

Q1. Explain Linux Operating System Architecture with a neat diagram.

Linux is a modular, open-source operating system based on UNIX principles. Its architecture follows a layered design that separates application space from kernel space to ensure stability, security, and performance. Understanding Linux architecture is essential for forensic investigations of Linux-based systems.

1. Layers of Linux Architecture

Linux architecture consists of the following main components:

A. User Space

1. Applications

These are user-executed programs such as:

- Web browsers
- Editors
- Compilers
- Scripts

Applications use system calls to request kernel services.

2. System Libraries (glibc, etc.)

Libraries provide:

- Standard functions
- Wrapper layers for system calls
- Interfaces for networking, file access, and I/O

Examples: **libc**, **libpthread**, **libm**

B. Kernel Space

The Linux kernel is the core component responsible for:

- Process management

- Memory management
- File system handling
- Device driver control
- Networking stack
- System security

It has direct access to hardware.

Kernel Subsystems Include:

1. Process Scheduler

Manages CPU allocation, priority, and multitasking.

2. Memory Manager

Handles paging, segmentation, and virtual memory.

3. Virtual File System (VFS)

Provides a uniform interface to all file systems like EXT4, XFS, BTRFS.

4. Device Drivers

Translate generic OS commands into hardware-specific operations.

5. Networking Stack

Implements TCP/IP, routing, firewall (iptables/nftables).

6. Inter-process Communication (IPC)

Includes pipes, shared memory, semaphores.

C. Hardware Layer

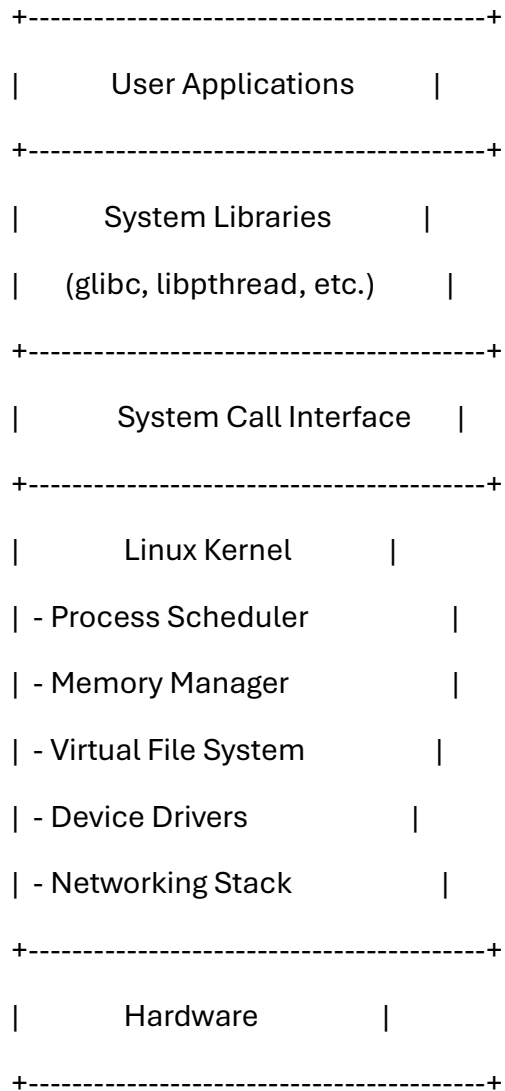
Includes:

- CPU
- RAM
- Storage devices
- Network interfaces

- Peripheral devices

The kernel interacts with hardware through drivers.

Neat Diagram (Copy in Exam)



Forensic Relevance

- Process forensics → understanding kernel scheduler
- Memory forensics → mapping virtual memory
- File system forensics → VFS behavior

- Log analysis → kernel messages (dmesg)
 - Rootkit detection → kernel module inspection
-

Conclusion

Linux architecture provides a robust, layered design that separates user and kernel operations. Its transparency, modularity, and strong logging mechanisms make Linux a highly analyzable system in forensic investigations.

Q2. Explain the EXT File System Architecture in detail.

The EXT family (EXT2, EXT3, EXT4) is one of the most widely used Linux file systems. It uses a block group-based architecture and provides reliability, journaling (EXT3/EXT4), and efficient storage management.

1. Basic Structure of EXT File System

The EXT file system consists of the following components:

A. Boot Block

Located in the first block of the partition.

Contains:

- Boot loader code
 - File system identification
-

B. Superblock

The most important metadata structure.

Stores information such as:

- Total number of inodes

- Total number of blocks
- Block size
- File system state
- Mount time, last write time
- UUID of file system

Located at a fixed offset and backed up in each block group.

C. Block Groups

EXT divides the file system into multiple block groups to reduce fragmentation and improve performance.

Each block group contains:

1. Block Bitmap – tracks free/used blocks
 2. Inode Bitmap – tracks free/used inodes
 3. Inode Table – stores inodes
 4. Data Blocks – store file content
-

D. Inodes

Each file in EXT is represented by an inode.

An inode contains:

- File type
- Permissions
- Owner and group ID
- Timestamps (Access, Modify, Change)
- File size
- Pointers to data blocks
- Extended attributes

E. Directory Structure

Directories are special files containing entries:

- Filename
- Corresponding inode number

F. Journaling (EXT3 & EXT4)

EXT3 and EXT4 maintain a journal to preserve consistency.

Journal logs:

- Metadata updates
- File system transactions
- Reduces corruption risks after crashes

2. Forensic Significance of EXT File System

A. Recovery of Deleted Files

Inode and block bitmaps reveal unused space where deleted data may still exist.

B. Timestamp Analysis

Inodes store:

- atime (access time)
- mtime (modified time)
- ctime (metadata change time)

EXT4 adds **creation time (ctime)**.

C. Journal Analysis

The journal may contain:

- Previous versions of metadata
- Partial file content

- Evidence of tampering or overwriting

D. Inode Table Analysis

Even after deletion, inode entries may retain metadata.

E. Block Group Backups

Superblock backups help recover corrupted partitions.

Conclusion

The EXT file system's robust metadata structure—including inodes, superblocks, block groups, and journaling—provides valuable forensic artifacts. Detailed analysis helps recover deleted files, reconstruct timelines, and detect tampering.

Q3. Explain how to recreate EXT partitions and analyze unallocated partitions in Linux.

Recreating EXT partitions and examining unallocated space is crucial in Linux forensics, especially when partitions are deleted, corrupted, or overwritten intentionally by attackers.

1. Recreating EXT Partitions

A. Step 1: Identify Partition Metadata

Analysts examine:

- Superblock
- Backup superblocks
- Block group descriptors

Use commands:

```
dumpe2fs /dev/sda1
```

```
mke2fs -n /dev/sda1 (shows backup locations)
```

B. Step 2: Locate Backup Superblocks

EXT maintains superblock backups at regular intervals (block groups).

Backup superblocks allow:

- Reconstruction of original file system
 - Recovery after corruption
-

C. Step 3: Analyze Block Group Descriptors

Block group descriptors contain:

- Block bitmap location
- Inode bitmap location
- Inode table location

From these, the file system layout can be reconstructed.

D. Step 4: Identify Inodes and Data Blocks

Using forensic tools, investigators extract:

- Inode tables
- Bitmaps
- Data blocks

Tools:

- Autopsy
 - Extundelete
 - Sleuth Kit
 - X-Ways
-

E. Step 5: Rebuild Partition Table

Tools like **TestDisk** can automatically detect EXT signatures and recreate the partition entries.

2. Analyzing Unallocated Partitions in Linux

Unallocated space may contain remnants of:

- Deleted files
- Previous installations
- Partial directory entries
- Old journal content

A. File Carving

Extracts files by searching for signatures.

Tools:

- Foremost
- PhotoRec
- Scalpel

B. Inode Analysis

Deleted inode entries may still contain:

- Ownership details
- Timestamps
- Block pointers

This helps reconstruct deleted files.

C. Journal Analysis (EXT3/EXT4)

The journal contains:

- Metadata transactions
- File creations/deletions

- Write logs

It may contain partial data from overwritten files.

D. Block Group Slack Analysis

Slack space within block groups may store previous file remnants.

E. Searching Unallocated Space

Tools scan raw partitions for:

- Strings
 - Emails
 - Chat logs
 - Binary signatures
-

Forensic Importance

A. Recovery of Deleted or Hidden Data

Even after deletion, EXT retains metadata until overwritten.

B. Detection of Anti-Forensics

Attackers may delete partitions; reconstruction uncovers intent.

C. Restoration of Corrupted Systems

Backup superblocks allow disaster recovery.

D. Timeline Reconstruction

Recovered inodes and journal entries help establish sequences of events.

Conclusion

Recreating EXT partitions and analyzing unallocated space allows investigators to recover critical evidence, reconstruct deleted file systems, and uncover intentional tampering. EXT's metadata-rich structure makes Linux forensics highly effective.

Q4. Explain various Linux log files and their forensic importance.

Linux maintains a wide range of log files that record system activities, user actions, authentication attempts, boot processes, service failures, and network events. These logs are essential in forensic investigations as they help reconstruct timelines and detect malicious activity.

Most Linux logs are stored in:

`/var/log/`

1. Important Linux Log Files

A. `/var/log/syslog` (or `/var/log/messages`)

Contains system-wide messages, including:

- Kernel events
- Service start/stop
- System errors
- Hardware activity

Forensic Value:

- Detects system failures
 - Tracks suspicious kernel messages
 - Shows module loading (potential rootkits)
-

B. `/var/log/auth.log` (or `/var/log/secure`)

Stores authentication-related events, including:

- Login attempts
- Logout events
- SSH connections

- sudo command usage
- Failed authentication attempts

Forensic Value:

- Identify brute-force attacks
 - Detect unauthorized access
 - Trace user privilege escalation
-

C. /var/log/kern.log

Contains logs from the Linux kernel.

Forensic Value:

- Driver-related issues
 - Kernel module tampering
 - Evidence of rootkits
-

D. /var/log/boot.log

Logs all messages generated during system boot.

Forensic Value:

- Identifies abnormal boot patterns
 - Detects unauthorized modifications to startup files
-

E. /var/log/dmesg

Contains kernel ring buffer messages.

Forensic Value:

- Hardware events
- Device connections
- Kernel warnings and errors

F. /var/log/faillog

Contains information on failed login attempts.

Forensic Value:

- Detects brute-force attacks
 - Shows repeated failed attempts
-

G. /var/log/lastlog

Stores last login time for each user.

Forensic Value:

- Detect unusual login times
 - Identify inactive or compromised accounts
-

H. /var/log/wtmp and /var/log/btmp

Binary logs storing login/logout events.

- **wtmp** → Successful logins
- **btmp** → Failed login attempts

Commands to view:

last

lastb

Forensic Value:

- Detect intruder sessions
 - Timeline reconstruction
-

I. /var/log/httpd/ or /var/log/nginx/

Web server logs containing:

- Access logs
- Error logs

Forensic Value:

- Tracks attacks like SQL injection, brute-force
 - Identifies attacker IP addresses
-

Conclusion

Linux logs form the backbone of forensic investigations. They provide evidence of authentication events, system behavior, network access, and malicious activity. Proper analysis allows investigators to reconstruct events with high accuracy.

Q5. Explain how to analyze important Linux artifacts related to user activity.

Linux stores many artifacts that reveal user actions, login activity, executed commands, opened files, and system interactions. These artifacts are essential for reconstructing behavior during forensic investigations.

1. User Login Artifacts

A. /var/log/auth.log

Contains:

- Successful logins
 - Failed logins
 - sudo usage
 - SSH connections
-

B. /var/log/wtmp, btmp, utmp

Access using commands:

last

lastb

who

Shows:

- User sessions
 - Login/logout times
 - TTY usage
-

C. ~/.bash_history

Stores commands executed by the user.

Forensic Use:

- Detect executed scripts
 - Identify data exfiltration commands
 - Trace privilege escalation attempts
-

2. User Home Directory Artifacts

A. SSH Keys in ~/.ssh/

- id_rsa
- known_hosts
- authorized_keys

These reveal:

- Remote systems accessed
 - Allowed inbound SSH connections
-

B. Cron Jobs

Locations:

- /etc/crontab
- /var/spool/cron/

Forensic Use:

- Detect persistence mechanisms
 - Identify scheduled malicious tasks
-

C. User Shell Configuration

Files:

- .bashrc
- .profile
- .bash_logout

These indicate:

- Script execution on login
 - Environment modifications
-

3. File System Artifacts

A. EXT File System Timestamps

Inodes store:

- atime
- mtime
- ctime
- ctime (EXT4)

These help build activity timelines.

B. Deleted File Recovery

Tools like:

- Sleuth Kit
- extundelete
can extract deleted files from unallocated space.

4. System Activity Artifacts

A. /var/log/syslog

Shows:

- System errors
- Service activity
- Application messages

B. Package Management Logs

Location varies by OS:

- Debian/Ubuntu → /var/log/apt/history.log
- RedHat/CentOS → /var/log/yum.log

Provides:

- Installed packages
- Updates
- Removed software

Useful for identifying unauthorized changes.

5. Command Execution Tracking

A. auditd Logs

Location: /var/log/audit/audit.log

auditd records:

- System calls
 - File access events
 - Privilege escalation
-

B. History File Timestamps

Using HISTTIMEFORMAT variable.

Conclusion

Linux contains numerous forensic artifacts that reveal user activity. Analyzing logs, shell history, SSH keys, cron jobs, inodes, and package logs allows investigators to reconstruct user actions and detect malicious behavior effectively.

Q6. Explain the analysis of Attached Devices in Linux systems and how to identify them for forensics.

Attached devices leave artifacts in Linux logs, kernel messages, and system directories. Forensic analysts examine these artifacts to identify USB storage devices, external disks, network adapters, and other peripherals connected to the system.

1. Kernel Logs (dmesg)

When a USB device is connected, Linux records messages such as:

- Device vendor ID
- Product ID
- Serial number
- Assigned device name (e.g., /dev/sdb)

Example:

```
dmesg | grep -i usb
```

Forensic Value:

- Reveals device identity
 - Shows connection and disconnection timestamps
-

2. /var/log/syslog or /var/log/messages

Contains:

- USB detection logs
- Mount events
- Device failures

Example log snippet:

usb 2-1: New USB device found, idVendor=0781, idProduct=5567

3. Mount History

Check /etc/mtab or /proc/mounts.

Shows:

- Device mounted
- Mount point
- Filesystem type

Example:

/dev/sdb1 on /media/usb type vfat

4. /dev Directory

Devices appear as:

- /dev/sda
- /dev/sdb
- /dev/ttyUSB0

Examining timestamps of device nodes helps correlate connection events.

5. Udev Information (/sys directory)

Each connected device has a directory under /sys:

Example:

/sys/block/sdb/device/

Contains:

- Vendor
 - Model
 - Serial number
-

6. lsusb Command

Output includes:

- Device manufacturer
- Product ID
- Bus number

Example:

lsusb

Bus 002 Device 004: ID 0781:5567 SanDisk Cruzer Blade

7. blkid Command

Provides filesystem details:

- UUID
 - Label
 - Partition type
-

8. Forensic Use Cases

A. Data Exfiltration Detection

USB device logs help determine if:

- Sensitive data was copied
 - Unauthorized devices were connected
-

B. Malware Spread

Malware may propagate via USB devices.

C. Timeline Reconstruction

Connection timestamps correlate with:

- Bash history
 - File access logs
 - Copy operations
-

D. Identifying Compromised Devices

Serial numbers help match USB evidence with physical devices.

Conclusion

Linux records extensive details about attached devices through kernel logs, syslog, udev, and mount history. These artifacts help investigators identify connected devices, reconstruct timelines, detect data theft, and correlate evidence with suspects.

Q7. Explain the analysis of Shared Locations in Linux and their forensic relevance.

Shared locations in Linux include directories and resources that multiple users or services can access. These locations often contain critical forensic artifacts related to user collaboration, file transfer, remote access, and unauthorized data sharing. Understanding these directories helps investigators detect data leakage, insider threats, and malicious activity.

1. Common Shared Locations in Linux

A. /home Directories

Each user has a separate home directory under /home, but within these directories users may create:

- Shared folders
- Public directories
- Symbolic links to shared areas

Forensic Importance:

- Reveals unauthorized file sharing
 - Indicates collaborative work environments
 - Contains chat logs, SSH keys, and browsing artifacts
-

B. /tmp and /var/tmp

Temporary directories accessible by all users.

Used for:

- File transfers
- Staging malicious scripts
- Compiling malware
- Temporary downloads

Forensic Importance:

- Malware often hides here
 - Attackers use /tmp for privilege escalation exploits
 - Deleted files may still be recoverable
-

C. /mnt and /media

Used for mounting external drives and network shares.

Forensic Importance:

- Shows evidence of external storage usage
 - Indicates whether USB devices or network shares were mounted
 - Prints mount timestamps from /var/log/syslog
-

D. /srv Directory

Stores data for services like:

- Web servers
- FTP
- NFS
- Samba

Forensic Importance:

- Reveals files served to external users
 - Useful in data exfiltration investigations
-

E. Network Shared Locations (NFS / Samba)

Network shares mounted via:

- /etc/fstab
- /proc/mounts

- /etc/samba/smb.conf

Forensic Importance:

- Shows remote file sharing
 - Tracks data copied across network drives
 - Identifies potential unauthorized access
-

2. Forensic Methods for Analyzing Shared Locations

A. Listing File Ownership and Permissions

Commands:

ls -l

getfacl

Reveals:

- Who can access which files
 - Permissions that may allow misuse
-

B. Examining Log Files

auth.log, syslog, and dmesg help track:

- File access
 - Mount operations
 - Permission changes
-

C. Investigating File Metadata

EXT inodes provide:

- Access time (atime)
- Modify time (mtime)
- Change time (ctime)

- Creation time (ctime – EXT4)

Useful to build activity timelines.

D. Recovery of Deleted Files

Shared folders often contain temporarily stored or deleted files that can be recovered from unallocated space.

Conclusion

Shared locations in Linux provide significant forensic insights into user collaboration, file sharing, and suspicious data transfers. By analyzing permissions, logs, metadata, and mounted locations, investigators can reconstruct user behavior and identify unauthorized activity.

Q8. Explain how to identify Recently Accessed Files, Programs, and Locations in Linux.

Linux systems store multiple artifacts that reveal recently accessed files, recently executed programs, and recently visited locations. These artifacts are critical for reconstructing user activity during forensic investigations.

1. Identifying Recently Executed Programs

A. ~/.bash_history

Contains list of commands executed by the user.

Forensic Use:

- Identifies executed scripts
 - Detects malicious commands
 - Tracks privilege escalation attempts
-

B. .zsh_history / fish_history

Stores command history for other shells.

C. /var/log/auth.log

Shows:

- sudo usage
- command authorization

Useful for linking commands to privileged users.

D. /var/log/audit/audit.log (auditd)

Provides:

- Detailed command execution
 - File access events
-

2. Identifying Recently Accessed Files

A. atime (Access Time) in Inodes

EXT inode stores:

- atime → when file was last opened

Use command:

stat filename

B. /proc//fd/

Lists open file descriptors for running processes.

C. File Manager Logs (GUI systems)

GNOME, KDE maintain:

- Recent files list
- Bookmarks
- Trash history

Example:

`~/.local/share/recently-used.xbel`

3. Identifying Recently Visited Locations

A. Shell Navigation Patterns

Commands:

`cd`

`pushd`

`popd`

`dirs`

These indicate folder movement.

B. Terminal History

Bash auto-logs:

- Commands used for navigation
- Paths accessed

Example:

`cd /var/www/html`

C. GUI File Browsers

Locations are stored in:

- ~/.config/gtk-3.0/bookmarks
 - ~/.local/share/nautilus/
 - ~/.cache/
-

4. Other Important Artifacts

A. Cron Logs

Reveal scheduled execution.

B. EXT Journal

Contains metadata of recently changed files.

C. Log Files

syslog, messages, and SSH logs reveal file access attempts.

Conclusion

Recently accessed files, executed programs, and visited locations can be identified using shell history, inode timestamps, GUI logs, process file descriptors, and Linux log files. These artifacts enable investigators to reconstruct user activity with high accuracy.

Q9. Explain how to analyze Installed Applications in Linux for forensic investigations.

Installed applications in Linux leave behind configuration files, logs, package metadata, binaries, dependencies, and service files that help investigators understand system usage and detect malicious software.

1. Identifying Installed Packages

Linux distributions use package managers that maintain detailed records.

A. Debian/Ubuntu

Commands:

`dpkg -l`

`apt list --installed`

Logs:

- `/var/log/apt/history.log`
- `/var/log/dpkg.log`

Shows:

- Installation date
- Package version
- Uninstall actions

B. RedHat/CentOS/Fedora

Commands:

`rpm -qa`

`yum history`

`dnf history`

Logs:

- `/var/log/yum.log`

C. Arch Linux

Command:

`pacman -Q`

2. Forensic Value of Package Logs

Package logs reveal:

- Recently installed or removed applications
 - Unauthorized software installations
 - Installation timestamps
 - Malicious packages added to system
-

3. Analyzing Application Configuration Files

Most applications store configs in:

/etc/

System-wide configs

Example:

- /etc/ssh/sshd_config
 - /etc/apache2/
-

~/.config/

User-specific app configurations.

Example:

- Browser settings
 - Chat applications
 - Remote desktop clients
-

4. Analyzing Application Logs

Applications generate logs under:

- /var/log/
- Application installation directories

Examples:

- Apache → /var/log/apache2/
- MySQL → /var/log/mysql/

Forensic Use:

- Detect unauthorized access
 - Trace user activity
 - Identify malware execution
-

5. Identifying Rogue or Malicious Applications

Indicators:

- Applications installed without admin approval
 - Services running at startup unexpectedly
 - Suspicious binaries in /usr/local/bin or /opt
 - Cron jobs that execute unknown programs
-

6. Examining Startup Scripts and Services

Use:

```
systemctl list-units --type=service
```

Check:

- Enabled services
 - Newly added daemons
 - Malware persistence mechanisms
-

Conclusion

Analyzing installed applications in Linux helps investigators identify legitimate and malicious software, review installation activities, inspect configuration files, and examine application logs. This analysis is crucial for reconstructing events, detecting intrusions, and identifying unauthorized software usage.

Q10. Explain how Linux logs different user activities and how these logs can be used in forensics.

Linux maintains detailed logs of user activities through multiple system components such as authentication services, shell history files, kernel messages, and system daemons. These logs help forensic investigators reconstruct timelines, detect suspicious activities, and identify unauthorized access.

1. Authentication Logs

File: /var/log/auth.log (Debian/Ubuntu)

File: /var/log/secure (RHEL/CentOS)

Logs include:

- Successful logins
- Failed login attempts
- SSH connections
- sudo command executions
- PAM authentication events

Forensic Importance:

- Detect brute-force attacks
 - Identify compromised accounts
 - Verify who logged in and when
-

2. Login Session Logs

Files:

- /var/log/wtmp → Successful logins
- /var/log/btmp → Failed logins
- /var/log/utmp → Current login sessions

Commands:

last

lastb

who

Forensic Use:

- Build user activity timelines
 - Track unauthorized access
-

3. Shell History Files

~/.bash_history, ~/.zsh_history, fish_history

Stores commands executed by users.

Forensic Use:

- Detect executed scripts
 - Identify privilege escalation attempts
 - Reveal data exfiltration commands
-

4. System Activity Logs

/var/log/syslog or /var/log/messages

Contains:

- Service start/stop
- Device activity
- Process crashes

Forensic Importance:

- Shows background activity
 - Detects suspicious system behavior
-

5. Kernel Logs

/var/log/kern.log

dmesg output

Contains:

- USB device connections
- Hardware events
- Kernel warnings

Forensic Use:

- Detect rootkit activity
 - Identify unauthorized device connections
-

6. Application Logs

Stored in /var/log/<application>/

Examples:

- Apache logs
- MySQL logs
- Mail server logs

Forensic Use:

- Identify web attacks
 - Trace DB modifications
 - Detect malicious sessions
-

7. Cron Logs

Location:

/var/log/syslog or /var/log/cron

Contains:

- Scheduled job execution
- Unauthorized cron entries

Forensic Use:

- Detect persistence mechanisms
 - Identify malicious scheduled tasks
-

Conclusion

Linux logs user authentication, file access, command execution, and system activity across various components. These logs provide crucial forensic insights that help detect intrusions, trace user actions, and reconstruct timelines.

Q11. Explain log analysis techniques in Linux and how they help identify user behavior, programs, and locations.

Linux log analysis involves examining system records to understand user activity, executed programs, file access patterns, and navigation behavior. Logs provide timestamped evidence essential for forensic investigations.

1. Techniques for Log Analysis

A. Keyword Searching

Using tools like:

grep

awk

sed

Search for:

- Usernames

- Commands
 - File paths
 - Error messages
-

B. Timeline Construction

Combine logs from:

- auth.log
 - syslog
 - bash history
 - journalctl
- to create a chronological sequence of events.
-

C. Pattern Analysis

Identify patterns such as:

- Repeated failed logins
 - Sudden privilege escalations
 - Unusual mount operations
-

D. Correlating Multiple Logs

Example:

- auth.log shows login
- bash_history shows executed commands
- syslog shows program errors

Correlation creates a full picture of user behavior.

E. Parsing Tools

Tools:

- Splunk
- Graylog
- ELK Stack
- GoAccess
- journalctl

These help visualize logs and detect anomalies.

2. Identifying User Behavior Through Logs

A. Authentication Behavior

auth.log shows:

- Login times
 - SSH sessions
 - sudo commands
-

B. Shell Command Behavior

History files show:

- Commands run
 - Scripts executed
 - File manipulations
-

C. Navigation Behavior

Logs reveal:

- Directories accessed
- Mounted drives

- Remote connections
-

3. Identifying Programs Using Logs

A. Syslog

Shows:

- Program start/stop
 - Errors
 - Crashes
-

B. systemd Logs (journalctl)

Provides detailed logs of:

- Services
 - Background processes
 - Boot processes
-

C. Application Logs

Examples:

- Apache logs reveal web activity
 - MySQL logs reveal database access
 - SSH logs reveal remote activity
-

4. Identifying Locations Accessed

A. File Access Logs

auditd logs record:

- Read events
- Write events

- File permission changes

B. EXT metadata

atime and mtime reveal file access patterns.

C. Terminal Navigation

Commands like:

```
cd /var/www/html
```

are stored in shell history.

Conclusion

Linux log analysis reveals detailed user behavior, executed programs, accessed files, and visited locations. By correlating logs across different components, investigators can reconstruct user actions and detect anomalies or malicious activity.

Q12. Explain the analysis of Installed Applications and Software Artifacts in Linux Forensics.

Linux systems maintain various logs, metadata, and configuration files that reveal installed applications, updates, package changes, and software behavior. Analyzing these artifacts is crucial for identifying unauthorized installations, malicious programs, and system modifications.

1. Identifying Installed Applications

A. Using Package Managers

Debian/Ubuntu

Commands:

```
dpkg -l
```

apt list --installed

Logs:

- /var/log/apt/history.log
 - /var/log/dpkg.log
-

RedHat/CentOS

Commands:

rpm -qa

yum history

dnf history

Logs:

- /var/log/yum.log
-

Forensic Use:

- Detect newly installed or removed programs
 - Identify malware added via package managers
 - Reconstruct installation timeline
-

2. Analyzing Application Directories

Locations:

- /usr/bin → Executables
- /usr/local/bin → User-installed programs
- /opt/ → Third-party software

Investigators check:

- File metadata
- Permissions

- Ownership
-

3. Analyzing Configuration Files

Most apps store configuration in:

- /etc/ (system-wide configs)
- ~/.config/ (user configs)
- ~/.local/share/

Forensic Use:

- Detect unauthorized app changes
 - Identify connection history (VPNs, browsers, messaging apps)
-

4. Analyzing Log Files of Applications

Examples:

- Apache → /var/log/apache2/
- NGINX → /var/log/nginx/
- MySQL → /var/log/mysql/

Logs contain:

- Requests
 - Errors
 - User access
 - Potential attacks
-

5. Detecting Rogue or Malicious Applications

Indicators:

- Unknown services running at startup
- Suspicious binaries found in /tmp or /opt

- New cron jobs executing malicious scripts
-

6. Software Metadata

Investigators examine:

- File hashes
- File signatures
- Modification timestamps
- Extended attributes

This reveals tampering or unauthorized updates.

Conclusion

Installed applications and their artifacts provide essential forensic insight into system activity, unauthorized changes, and malicious installations. By examining package logs, application directories, configurations, and logs, investigators can reconstruct software behavior and identify security breaches.
